

Mongo ATLAS

07/10/2025

Uso de JSON

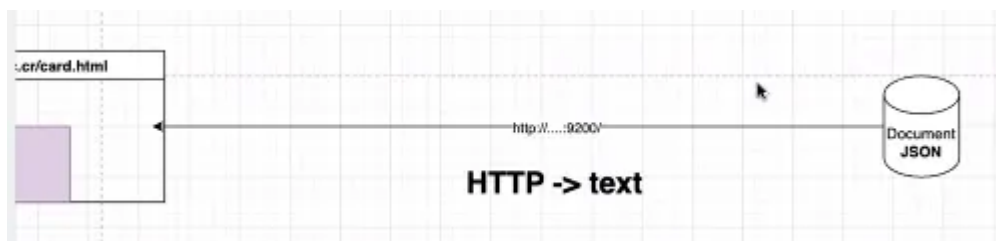
Json es el formato que utiliza JS para intercambiar objetos. Antes se ocupaba aplicar parseo, para traducir entre lenguajes. Aumentando la carga entre las bases de datos.

1. Permite tener un almacenamiento de datos en disco
2. Transmitir datos sin necesidad de tantas transformaciones y con ello se reduce el uso de cpu.

Las bases de datos pueden tener un lenguaje de intercambio de datos → Las bases de datos utilizan por ejemplo un mecanismo binario para intercambio de datos, toma el sql lo compila en binario y se le envia al destino el mismo lo traduce a una representacion interna del lenguaje de programacion o de la base de datos.

Es decir, existe etapas de parsing de documentos en los lenguajes de programacion.

Decidieron usar los web services como lenguaje de intercambio de datos y convertirlo en el lenguaje que usa la web. El protocolo HTTP es un lenguaje en texto con caracteres visibles de la tabla ascii. Con ello termina siendo un documento JSON.



Document Model

Corazon de la bases de datos documentales. Basados en modelos key-values, son campos únicos que representan la base de datos.

La base de datos documental se basa en el document model, y el se encarga de hacer la representación en formato JSON.

1. Se puede modificar la estructura del documento y modificar disarmaments.
2. Los documentos pueden tener diferentes campos
3. Se puede representar cualquier tipo de dato, se tiene una cuota superior de 16 MB los docs mayores a 16MB no se pueden almacenar dentro de mongo.
4. Cuando se tiene volúmenes grandes se suele tener datos binarios (Imágenes, videos, audios, archivos ejecutables, lagun tipo de archivo custom (zip, targ)) cualquier archivo que no se pueda leer en texto plano.

1. Cuando se tiene datos binarios en un json, se puede convertir el dato en Base 64. Tomar una representación binaria y convertirlo en una representación visual en texto, es decir, codificar la información en texto, guardarlo en los 64 caracteres visibles de la tabla ascii. Es decir, una codificación con menos caracteres.



Mongo va a guardar los datos como string, y si se desea buscar esos datos Mongo no los va a identificar.

GridFS

Permite guardar archivos y datos grandes en un file system.

Object Storage

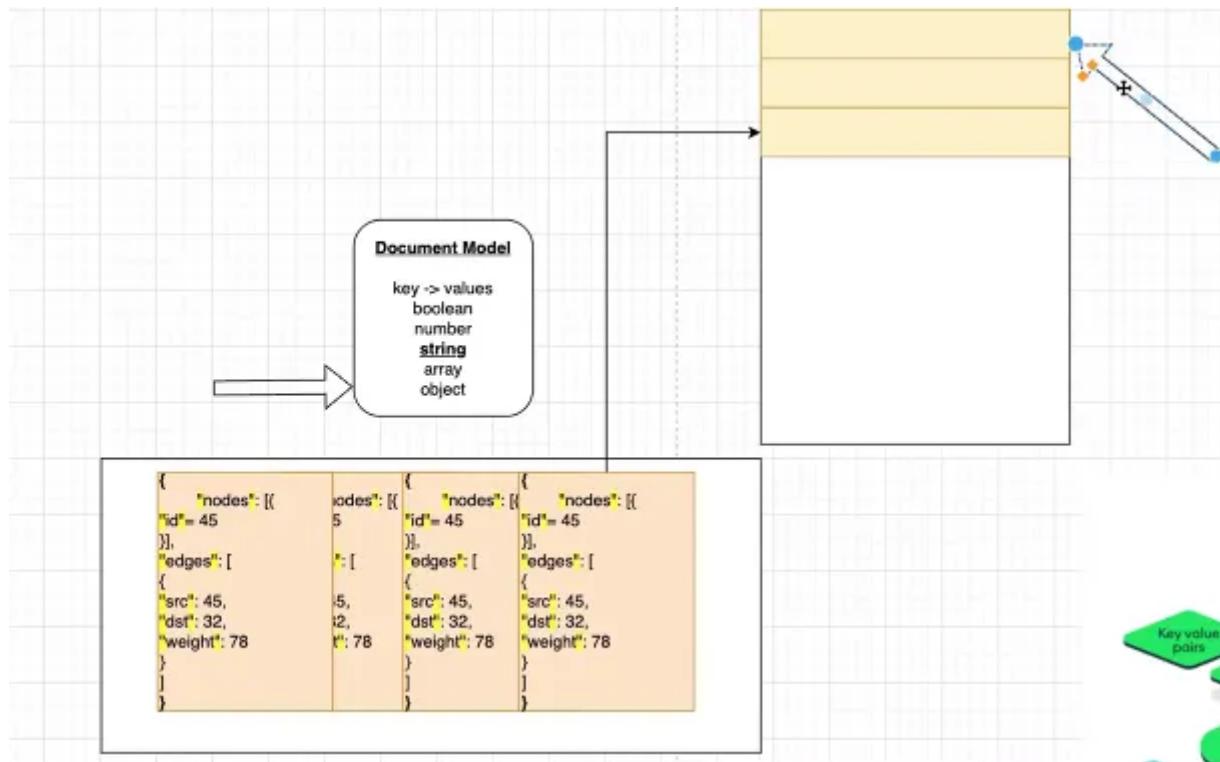
Almacenamiento de documentos, como un S3.

Al tener datos de json que contiene binarios, se divide los datos binarios y los json. Para unirlos, el binario se guarda en un object storage que nos genera un URL, y el URL se guarda en el documento JSON, es decir teniendo una referencia de donde se guarda el documento. Con ello, los documentos van a tener fields sencillos de buscar.

Es importante tomar en cuenta los caracteres que contienen los documentos, pues los mismos se traducen en bytes y se deben almacenar. Puede generar desperdicio de almacenamiento. Al tener los archivos uno por uno en un folder es ineficiente para la base de datos.

Alternativa

Agarrar un solo archivo donde se colocan los documentos json uno tras del otro, conectados al siguiente. Cuando se ocupe recuperar docs se abre un solo solo archivo y se mueve sobre el archivo.



Cada uno de los documentos van a ser un JSON, terminan y empiezan con llaves. Por ello se tiene que contruir un metodo que recorra el archivo e identifique donde empieza y termina cada documento.

SCAN

Puede ser leyendo fila por fila o bytes por bytes interpretando los datos hasta encontrar lo que se buscaba. Es el método menos deseado y en el peor de los casos esta de ultimo.

BJSON / Binary JSON

Es un documento que suelen ser grandes. Sucede cuando a la hora de construir el documento, se busca un forma en la cual se pueda convertir el doc en una representacion de documento mas amigable.

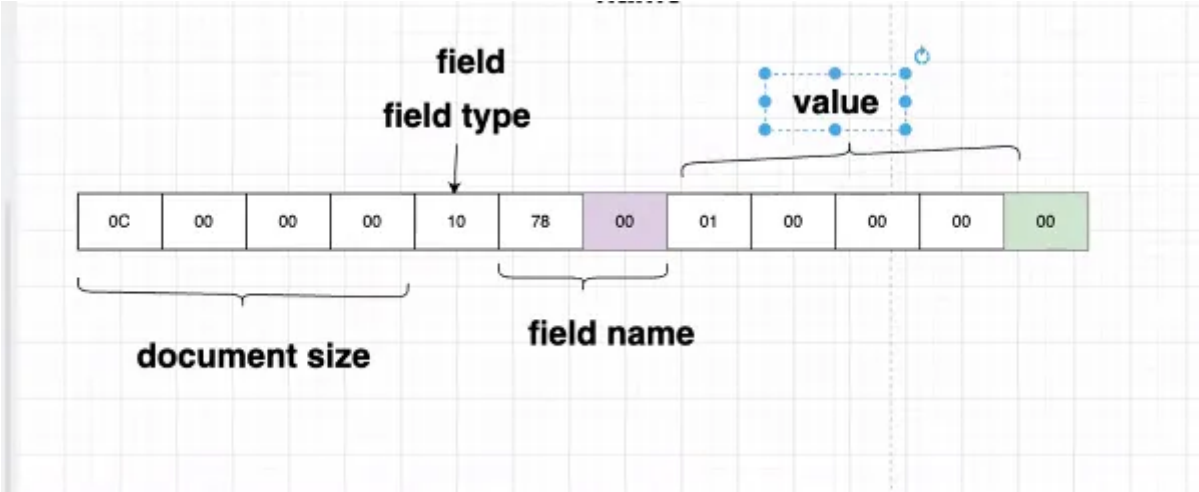
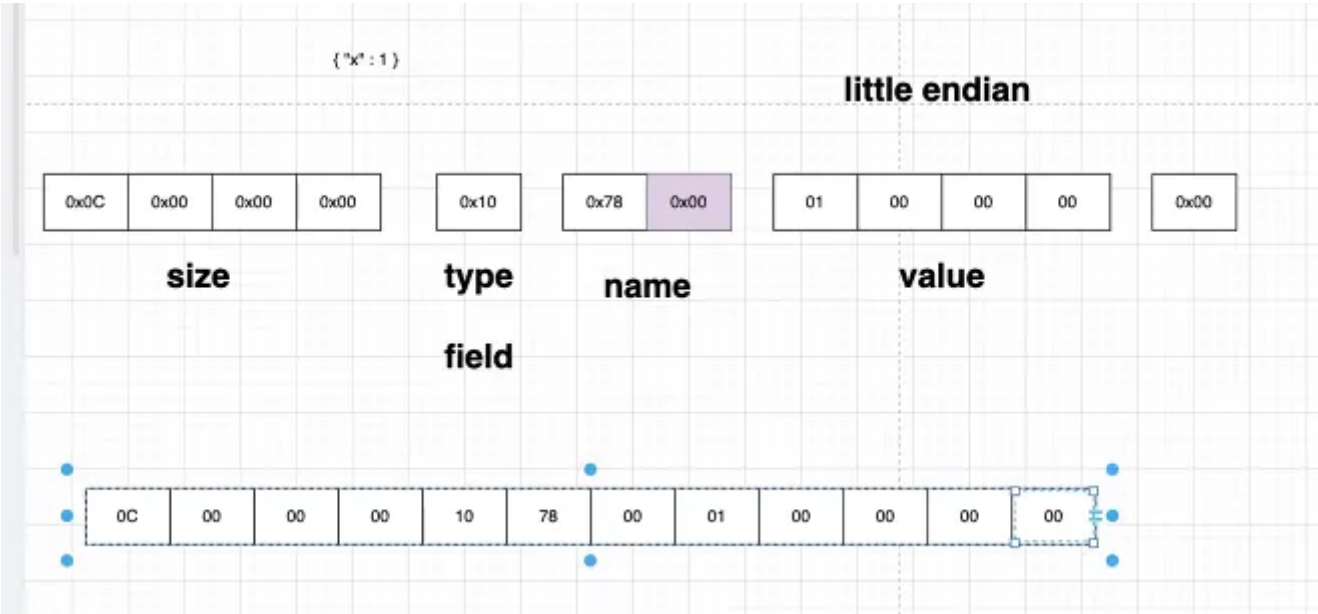
Codificacion de un documento JSON

En BJSON se tendra 4 bytes que respresentan el tamaño del registro. Es decir, convierto el doc en un registro.

Tendra un 1 byte que representa el tipo de field.

Tendra 2 bytes que representa el name del field. El nombre de un cmapo termina siempre en 0x00 el cual es el terminador de documento

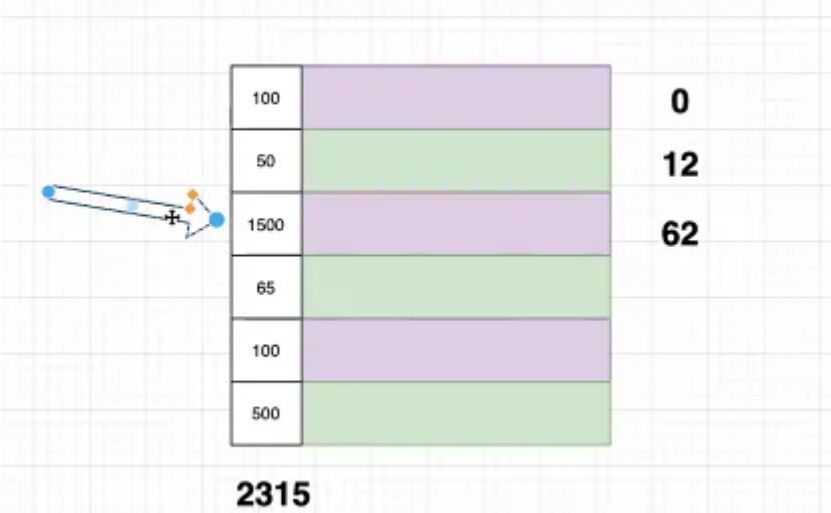
Tendra 4 bytes para representar el valor del campo. A bajo nivel se maneja en little endian.



El verde representa un terminador.

Esto facilita pues el manejador del almacenamiento de Mongo, lee los caracteres necesarios. Moviendo el puntero del disco de 12 en 12 caracteres por ejemplo, esto segun la necesidad de la posición del manejador.

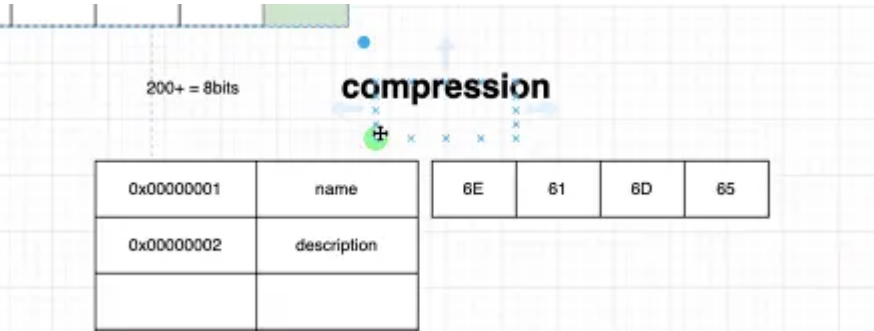
Haciendo los saltos mas eficientes, pues no revisa byte por byte para saber donde inician los documentos. Pues se codifican todos con un tamaño fijo.



Nombre del campo

Al ser datos largos, mongo aplica una simple compresion de los mismos. Con ello, codifica los nombres para tener un nivel de compresion en el tamano de los field name.

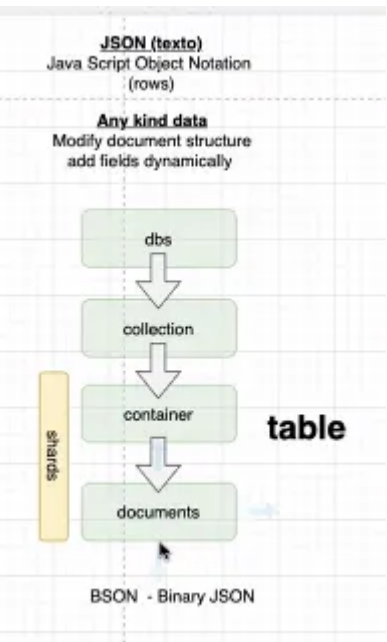
- 1. Con el uso de codificacion se pasara de tener 4 campos a solo 1 campo.
- 2. Reduce la cantidad de info y la cantidad de veces que se accede a buscar informacion, pues se tiene una tabla que resume toda la informacion.



El bianary JSON se usa a nivel de motor de base de datos. En mongo se usa wiredtiger/wiretiger. El mismo se encarga de manejarlo, organizarlo etc. Este es un manejador de mongo que consume mucha memoria, pues intenta mantener toda la info en memoria principal para reducir las entradas a disco, es por ello que se debe configurar bien para evitar que se infle de memoria principal.

Los datos binarios no son sencillos para realizar busquedas. Cuando se tiene una imagen se debe pasar por un OCR (Hace el reconocimiento de los documentos) , se extrae el texto que hay en la imagen, se puede obtener el url y ademas de guarda en el docuemento el texto extraido para cuando haya procesos de busqueda puedan haber busqueda sobre imagenes.

Cuando se habla de videos funciona igual pues un video es un conjunto de frames/imagenes. La diferencia es que se tiene el tiempo en que el texto aparece por medio de metadatos (datos de los datos).

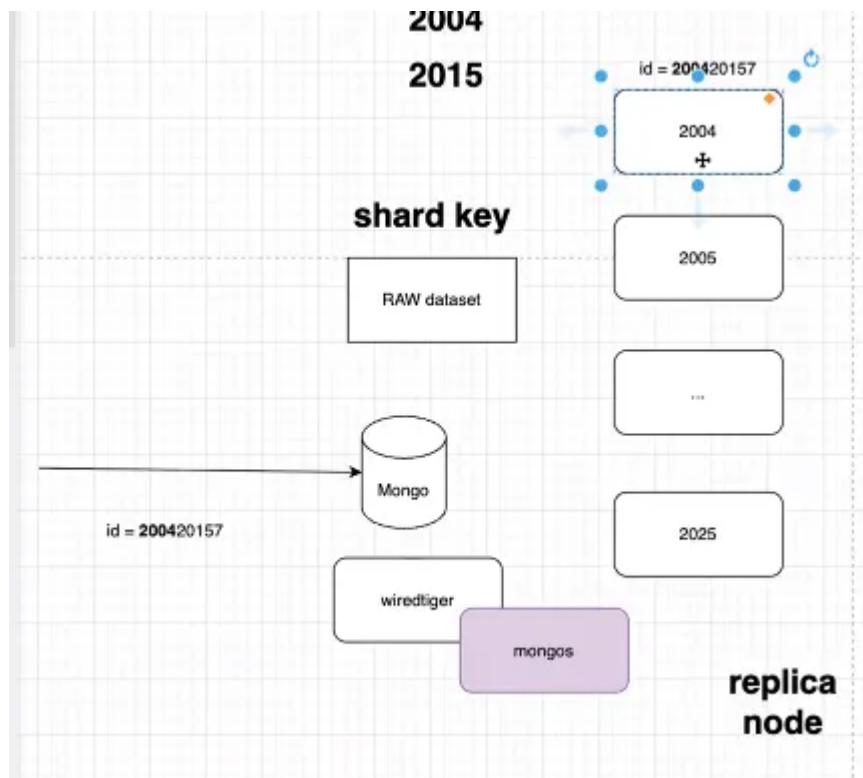


Mongo a referirse a una bd se habla de coleccion, tabla se habla de contendor, row se habla de documento.

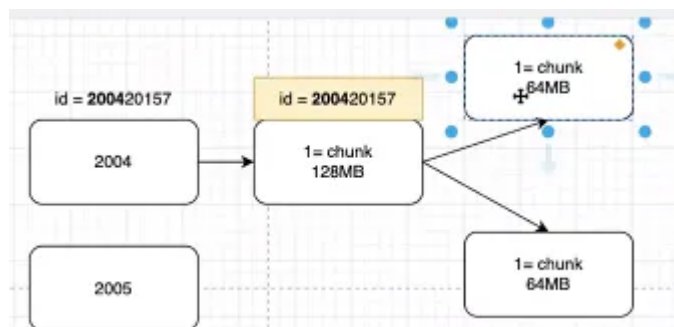
RAW Dataset

Permite dividir los datos.

Se maneja shard key → define como una unidad se rutea o se guardan los datos. Se les establece id a los documentos cuando mongo ops recibe el podra saber en que shard guardarlo, los shard son replicas que se guardan en nodos de mongo.



Se les asigna chunk, que tienen un limite de tamaño. Automaticamente el wiredtiger divide el chunk en dos si este sobrepasa el limite de tamaño. Se base en la cardinalidad de los datos con subdivision de los mismos con



el mismo shardkey del principal.

Entre mas chunk se tenga va a ser mas facil leerlo completo y subirlo a memoria.

JUMBO

Es un chunk muy grande que quedo asi pues el shard key no le permitio hacerle la subdivision de los datos.

Cuando se ve un shard que no esta creciendo mucho, mongo analiza si se puede hacer un reshard, y mover chunks para enviarlo a shards que no estan en uso.

Si el shard se hace muy grande, las consultas van a redirigirse a esos nodos buscando recursos y los pequenos van a estar desperdiciando recursos.

Es decir, se hacen un balanceo de shards.

Tipos de servidores

Replica Set

No se tiene el concepto de shard. Es decir, cada uno de los servidores van a tener el 100% de los datos, siendo copias idénticas de los mismo.

Replica Set Sharded

Se puede escoger cuanta replica de los datos por shard se pueden tener por servidor. Se puede definir la porcion de los datos que tendra cada shard, aumenta velocidad pero pierde disponibilidad.

